SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2&3)		
Student Name:	HEMASRINITHA P	**Reg. No.:**	192572179
Section / Batch:		**Date:**	06-11-2026

Code of Conduct

*I **HEMASRINITHA P (192572179)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Hemasrinitha P

Instructions

- Read each question carefully before answering.
- Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

S.No.	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
					TOTAL MARKS	

QUESTIONS

Question 1: Regular Expression-Based Text Processing

Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

Question 2: Finite State Automata for Word Pattern Recognition

Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

Question 3: Morphological Analysis of English Words

Develop a Python program to perform morphological analysis on a given set of English words.

1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

Question 4: Inflectional and Derivational Morphology

Write a Python program to identify and classify inflectional and derivational word forms from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.
4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

Question 5: Porter Stemming Algorithm

Implement the Porter Stemming Algorithm using Python to identify the root forms of English words.

1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.
5. Analyze how stemming reduces different word forms to a common base.

Example words: playing, played, studies, studying, connected, connection.

Question 6: Implementation of N-gram Language Models

Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

Question 7: Word Frequency and N-gram Smoothing

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

Question 8: Entropy Calculation for Language Models

Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.
5. Analyze how the context of preceding words influences the uncertainty of the next word.

Question 9: English Part-of-Speech Tagging

Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of each POS category.

Question 10: Comparative Analysis of POS Tagging Techniques

Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.

Question	Major Concept	BTL Level	Marks
1	Regular Expressions	3	10
2	Finite State Automata	3	10
3	Morphological Analysis	4	10
4	Inflectional and Derivational Morphology	3	10
5	Porter Stemmer	4	10
6	N-gram Models	3	10
7	Word Counting and Smoothing	3	10
8	Entropy	4	10
9	POS Tagging	3	10
10	POS Tagging Techniques	4	10

Evaluation Rubric (10 Marks per Assignment)

Evaluation Criteria	Marks
Understanding of the NLP Concept	2
Python Implementation and Logic	3
Correctness of Output	2
Analysis and Interpretation of Results	2
Code Readability and Documentation	1
Total	10

QUESTION 1

AIM

To develop a Python program using regular expressions to extract email addresses, phone numbers, capitalized words, numerical values, and words ending with ing and ed from a user-given paragraph.

CONCEPT

Regular Expression (RegEx) is a technique used to identify and extract specific patterns from text. Python provides the re module to perform pattern matching.

- re.findall() – Extracts all matching patterns.
- \d – Matches digits.
- \w – Matches letters, digits and underscores.
- \b – Represents a word boundary.
- + – One or more occurrences.
- * – Zero or more occurrences.
- [A-Z] – Matches uppercase letters.
- [a-z] – Matches lowercase letters.

PROGRAM

```
import re
text = input("Enter a paragraph: ")
emails = re.findall(r'\b[\w.-]+@[\w.-]+\.\w+\b', text)
phones = re.findall(r'\b\d{10}\b', text)
capital_words = re.findall(r'\b[A-Z][a-z]*\b', text)
numbers = re.findall(r'\b\d+\b', text)
ing_words = re.findall(r'\b\w+ing\b', text)
ed_words = re.findall(r'\b\w+ed\b', text)
print("\nEmail Addresses:", emails)
print("Phone Numbers:", phones)
print("Capitalized Words:", capital_words)
print("Numerical Values:", numbers)
print("Words ending with ing:", ing_words)
print("Words ending with ed:", ed_words)
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Py
thon313/NLP AssignemntProg-1.py
Enter a paragraph: John is studying Python and learning NLP. He
scored 95 marks and contacted teacher@example.com. Call him at
9876543210. Mary is playing and completed her work.

Email Addresses: ['teacher@example.com']
Phone Numbers: ['9876543210']
Capitalized Words: ['John', 'Python', 'He', 'Call', 'Mary']
Numerical Values: ['95', '9876543210']
Words ending with ing: ['studying', 'learning', 'playing']
Words ending with ed: ['scored', 'contacted', 'completed']
```

ANALYSIS AND INTERPRETATION

1. Email addresses and phone numbers are extracted using specific patterns.
2. Capitalized words are identified using uppercase letter matching.
3. Numerical values are extracted using digit patterns.
4. Words ending in ing and ed are identified using suffix matching.
5. Each extracted category is displayed separately for easy analysis.

RESULT

Thus, the program successfully extracts all the specified language features from the user-given paragraph using regular expressions.

QUESTION 2

AIM

To implement a Finite State Automaton (FSA) using Python to recognize words starting with vowels, ending with consonants, and containing consecutive vowels.

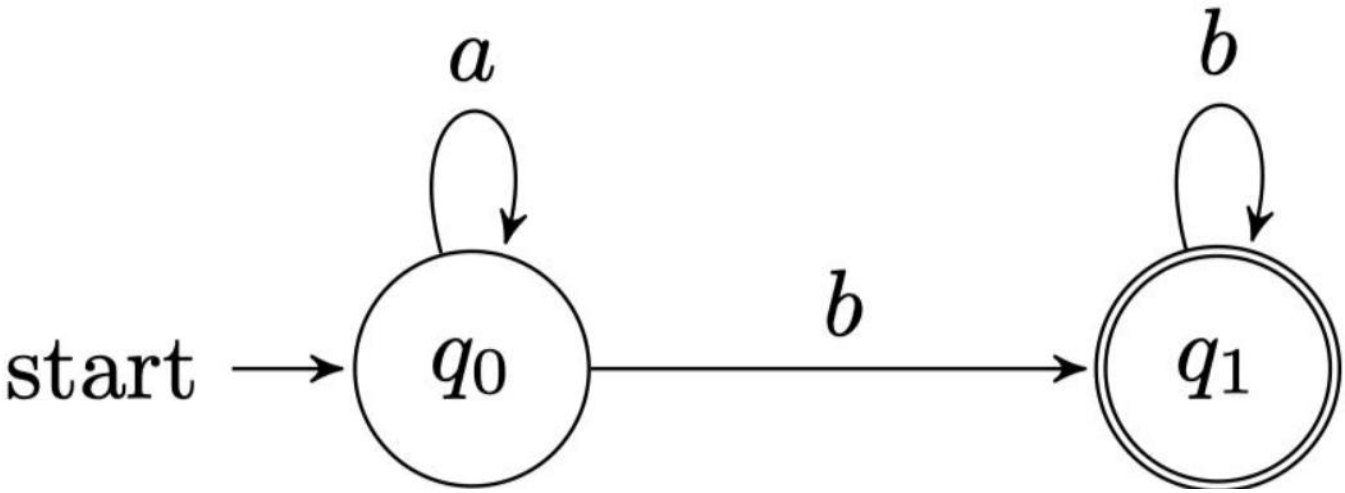
CONCEPT

A Finite State Automaton (FSA) is a mathematical model used to recognize patterns in strings. It consists of states, transitions, an initial state, and final states.

The given program checks three conditions:

1. Starting vowel – The word must begin with a, e, i, o, or u.
2. Ending consonant – The word must end with a consonant.
3. Consecutive vowels – The word must contain two vowels next to each other.

A word is accepted only when all three conditions are satisfied.



Where,

- q0: Initial state where processing begins.
- Input: Symbol or character read by the automaton.
- q1: Intermediate state after reading input.
- q2: Final (accepting) state.
- Transition: Arrow showing movement between states.
- Reject state: Indicates that the input is not accepted.

PROGRAM

```
def check_word(word):
    vowels = "aeiou"
    starts_vowel = word[0].lower() in vowels
    ends_consonant = word[-1].lower() not in vowels
    consecutive_vowels = any(
        word[i].lower() in vowels and word[i+1].lower() in vowels
        for i in range(len(word)-1)
    )
    print("\nWord:", word)
    print("Starts with vowel:", starts_vowel)
    print("Ends with consonant:", ends_consonant)
    print("Contains consecutive vowels:", consecutive_vowels)
    if starts_vowel and ends_consonant and consecutive_vowels:
        print("Accepted")
    else:
        print("Rejected")
words = input("Enter words separated by spaces: ").split()
for word in words:
    check_word(word)
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Py
thon313/NLP AssignemntProg-2.py
Enter words separated by spaces: apple orange eagle ant

Word: apple
Starts with vowel: True
Ends with consonant: False
Contains consecutive vowels: False
Rejected
```

```
Word: orange
Starts with vowel: True
Ends with consonant: False
Contains consecutive vowels: False
Rejected

Word: eagle
Starts with vowel: True
Ends with consonant: False
Contains consecutive vowels: True
Rejected

Word: ant
Starts with vowel: True
Ends with consonant: True
Contains consecutive vowels: False
Rejected
```

ANALYSIS AND INTERPRETATION

1. The program checks whether each word begins with a vowel.
2. It verifies whether the last character is a consonant.
3. It searches for consecutive vowels within each word.
4. Words satisfying all three conditions are accepted.
5. Words failing any condition are rejected.

RESULT

Thus, the Python program successfully implements an FSA to identify word patterns based on the given vowel and consonant conditions.

QUESTION 3

AIM

To develop a Python program to perform morphological analysis by identifying root words, prefixes, suffixes, and morphological structures of English words.

CONCEPT

Morphological analysis is the process of studying the internal structure of words by identifying their root words, prefixes, and suffixes.

A root is the main part of a word that carries its basic meaning. A prefix is added before the root, while a suffix is added after the root.

For example, the word unhappiness consists of the prefix un-, root happy, and suffix -ness.

Morphological analysis helps computers understand how words are formed and how their meanings change.

PROGRAM

```
words = ["unhappiness", "replaying", "disagreement", "careless"]
prefixes = ["un", "re", "dis"]
suffixes = ["ness", "ing", "ment", "less"]
for word in words:
    root = word
    prefix = ""
    suffix = ""
    for p in prefixes:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break
    for s in suffixes:
        if root.endswith(s):
            suffix = s
            root = root[:-len(s)]
            break
```

```
    print("\nWord:", word)
    print("Prefix:", prefix if prefix else "None")
    print("Root:", root)
    print("Suffix:", suffix if suffix else "None")
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppD
ata/Local/Programs/Python/Python313
/NLP AssignemntProg-3.py

Word: unhappiness
Prefix: un
Root: happi
Suffix: ness

Word: replaying
Prefix: re
Root: play
Suffix: ing

Word: disagreement
Prefix: dis
Root: agree
Suffix: ment

Word: careless
Prefix: None
Root: care
Suffix: less
```

ANALYSIS AND INTERPRETATION

- un, re, and dis are prefixes.
- ness, ing, ment, and less are suffixes.
- The root represents the main meaning of the word.
- In unhappiness, the original base is happy; its spelling changes to happi when -ness is added.

RESULT

Thus, the Python program successfully identifies the prefixes, root words, and suffixes of the given English words using morphological analysis.

QUESTION 4

AIM

To develop a Python program to identify and classify inflectional and derivational word forms from a given text.

CONCEPT

Inflectional and derivational morphology are two types of word formation.

- **Inflectional morphology** changes the grammatical form of a word without changing its basic meaning or word class. Examples include adding -s for plurals, -ed for past tense, and -ing for continuous actions.
- **Derivational morphology** creates a new word or changes its meaning or grammatical category by adding prefixes or suffixes. Examples include un-, re-, -ness, and -ful.

PROGRAM

```
text = input("Enter a sentence: ")
words = text.lower().split()
inflectional = ["s", "es", "ed", "ing"]
derivational = ["un", "re", "dis", "ness", "less", "ful"]
print("\nInflectional Words:")
for word in words:
    if any(word.endswith(s) for s in inflectional):
        print(word)
print("\nDerivational Words:")
for word in words:
    if any(word.startswith(p) or word.endswith(s)
           for p in ["un", "re", "dis"]
           for s in ["ness", "less", "ful"]):
        print(word)
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python3
13/NLP AssignemntProg-4.py
Enter a sentence: The students are playing games. John completed his
work. She is unhappy and careless.

Inflectional Words:
students
playing
completed
his
is

Derivational Words:
unhappy
```

ANALYSIS AND INTERPRETATION

- Plural nouns are formed by adding -s or -es (e.g., cat → cats).
- Past-tense verbs are formed by adding -ed (e.g., play → played).
- Present participles are formed by adding -ing (e.g., play → playing).
- Prefixes such as un-, re-, and dis- change the meaning of words (e.g., happy → unhappy).
- Suffixes such as -ness, -ful, and -less create new words (e.g., kind → kindness).
- Inflectional morphology changes grammatical forms, while derivational morphology creates new words or changes their meaning or grammatical category.

RESULT

Thus, the Python program successfully identifies and classifies inflectional and derivational word forms from the given text.

QUESTION 5

AIM

To implement the Porter Stemming Algorithm using Python to identify the root forms of English words and compare original words with their stemmed forms.

CONCEPT

Porter Stemming is a text-processing technique used in Natural Language Processing (NLP) to reduce words to their root form by removing common prefixes and suffixes.

It uses a set of predefined rules to convert different forms of a word into a common stem. For example, playing, played, and plays are reduced to play.

The Porter Stemmer algorithm is widely used in information retrieval, search engines, and text classification to reduce word variations and simplify text processing.

PROGRAM

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
text = input("Enter a paragraph: ")
ps = PorterStemmer()
words = word_tokenize(text.lower())
stems = {}
print("\nOriginal Word\tStemmed Word")
for word in words:
    if word.isalpha():
        stem = ps.stem(word)
        print(word, "\t\t", stem)
        stems.setdefault(stem, []).append(word)
print("\nWords with Same Stem:")
for stem, original in stems.items():
    if len(original) > 1:
        print(stem, ":", original)
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/NLP Assig
nemntProg-5.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a paragraph: Playing played studies studying connected connection

Original Word   Stemmed Word
playing                 play
played           play
studies                 studi
studying                studi
connected               connect
connection              connect

Words with Same Stem:
play : ['playing', 'played']
studi : ['studies', 'studying']
connect : ['connected', 'connection']
```

ANALYSIS AND INTERPRETATION

- The Porter Stemmer removes common suffixes from words to generate their stems.
- Words such as playing, played, and plays are reduced to play.
- Words such as studies and studying are reduced to studi.
- Different forms of the same word are identified and grouped using their common stems.
- This reduces vocabulary size and eliminates unnecessary word variations.
- The results show that stemming simplifies text processing and supports efficient information retrieval and text analysis.

RESULT

Thus, the Porter Stemming Algorithm successfully reduces different English word forms to their common stem.

QUESTION 6

AIM

To develop a Python program to implement unigram, bigram, and trigram language models, calculate their frequencies and probabilities, and compare the results.

CONCEPT

N-gram models are statistical language models used in Natural Language Processing (NLP) to predict and analyze sequences of words.

An N-gram is a sequence of N consecutive words in a text. There are three common types:

- Unigram: Single word (e.g., I)
- Bigram: Two consecutive words (e.g., I love)
- Trigram: Three consecutive words (e.g., I love NLP)

N-gram models use word frequencies and probabilities to understand language patterns and predict the next word.

PROGRAM

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from collections import Counter
import nltk
nltk.download('punkt_tab')
text = input("Enter a paragraph: ")
words = word_tokenize(text.lower())
unigram = Counter(words)
bigram = Counter(ngrams(words, 2))
trigram = Counter(ngrams(words, 3))
print("\nUnigram Frequencies:", unigram)
print("\nBigram Frequencies:", bigram)
print("\nTrigram Frequencies:", trigram)
print("\nBigram Probabilities:")
for bg, count in bigram.items():
    probability = count / unigram[bg[0]]
    print(bg, ":", round(probability, 3))
```

```
print("\nTrigram Probabilities:")
for tg, count in trigram.items():
    probability = count / bigram[tg[:2]]
    print(tg, ":", round(probability, 3))
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/NLP Assig
nemntProg-6.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a paragraph: I love natural language processing. I love learning Python. I
 love NLP.

Unigram Frequencies: Counter({'i': 3, 'love': 3, '.': 3, 'natural': 1, 'language
': 1, 'processing': 1, 'learning': 1, 'python': 1, 'nlp': 1})

Bigram Frequencies: Counter({('i', 'love'): 3, ('.', 'i'): 2, ('love', 'natural'
): 1, ('natural', 'language'): 1, ('language', 'processing'): 1, ('processing',
'.'): 1, ('love', 'learning'): 1, ('learning', 'python'): 1, ('python', '.'): 1,
 ('love', 'nlp'): 1, ('nlp', '.'): 1})

Trigram Frequencies: Counter({('.', 'i', 'love'): 2, ('i', 'love', 'natural'): 1
, ('love', 'natural', 'language'): 1, ('natural', 'language', 'processing'): 1,
('language', 'processing', '.'): 1, ('processing', '.', 'i'): 1, ('i', 'love', '
learning'): 1, ('love', 'learning', 'python'): 1, ('learning', 'python', '.'): 1
, ('python', '.', 'i'): 1, ('i', 'love', 'nlp'): 1, ('love', 'nlp', '.'): 1})

Bigram Probabilities:
('i', 'love') : 1.0
('love', 'natural') : 0.333
('natural', 'language') : 1.0
('language', 'processing') : 1.0
('processing', '.') : 1.0
('.', 'i') : 0.667
('love', 'learning') : 0.333
('learning', 'python') : 1.0
('python', '.') : 1.0
('love', 'nlp') : 0.333
('nlp', '.') : 1.0

Trigram Probabilities:
('i', 'love', 'natural') : 0.333
('love', 'natural', 'language') : 1.0
('natural', 'language', 'processing') : 1.0
('language', 'processing', '.') : 1.0
('processing', '.', 'i') : 1.0
('.', 'i', 'love') : 1.0
('i', 'love', 'learning') : 0.333
('love', 'learning', 'python') : 1.0
('learning', 'python', '.') : 1.0
('python', '.', 'i') : 1.0
('i', 'love', 'nlp') : 0.333
('love', 'nlp', '.') : 1.0
```

ANALYSIS AND INTERPRETATION

- Unigrams identify the frequency of individual words.
- Bigrams identify frequently occurring pairs of consecutive words.
- Trigrams identify patterns of three consecutive words.
- Higher-order N-grams provide more contextual information.
- Word probabilities help estimate the likelihood of a sequence.
- N-gram models are useful in text prediction, speech recognition, machine translation, and language processing.

RESULT

Thus, the program successfully generates unigram, bigram, and trigram models and calculates their frequencies and probabilities.

QUESTION 7

AIM

To develop a Python program to calculate word frequencies, generate bigram frequencies, calculate unsmoothed probabilities, and implement Add-One (Laplace) smoothing.

CONCEPT

Word frequency measures how often each word occurs in a text, while N-gram frequency counts consecutive word sequences.

N-gram smoothing is a technique used to handle words or word sequences that do not appear in the training text. Add-One (Laplace) smoothing adds 1 to every N-gram count to avoid zero probabilities.

The probability formulas are:

- Unsmoothed: Count of N-gram / Count of previous word
- Laplace smoothing: (N-gram count + 1) / (Previous word count + Vocabulary size)

PROGRAM

```
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter
import nltk
nltk.download('punkt_tab')
print("Enter a paragraph (press Enter twice to finish):")
lines = []
while True:
    line = input()
    if line == "":
        break
    lines.append(line)
text = " ".join(lines).lower()
words = word_tokenize(text)
unigram = Counter(words)
bigram = Counter(ngrams(words, 2))
vocab = len(unigram)
print("\nWord Frequencies:", unigram)
print("\nBigram Frequencies:", bigram)
```

```
print("\nBigram Probabilities:")
for bg, count in bigram.items():
    print(bg, ":", round(count / unigram[bg[0]], 3))
print("\nLaplace Smoothed Probabilities:")
for bg, count in bigram.items():
    probability = (count + 1) / (unigram[bg[0]] + vocab)
    print(bg, ":", round(probability, 3))
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/NLP Assig
nemntProg-7.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a paragraph (press Enter twice to finish):
I love natural language processing.
I love learning Python.
I love NLP.


Word Frequencies: Counter({'i': 3, 'love': 3, '.': 3, 'natural': 1, 'language':
1, 'processing': 1, 'learning': 1, 'python': 1, 'nlp': 1})

Bigram Frequencies: Counter({('i', 'love'): 3, ('.', 'i'): 2, ('love', 'natural'
): 1, ('natural', 'language'): 1, ('language', 'processing'): 1, ('processing',
'.'): 1, ('love', 'learning'): 1, ('learning', 'python'): 1, ('python', '.'): 1,
 ('love', 'nlp'): 1, ('nlp', '.'): 1})

Bigram Probabilities:
('i', 'love') : 1.0
('love', 'natural') : 0.333
('natural', 'language') : 1.0
('language', 'processing') : 1.0
('processing', '.') : 1.0
('.', 'i') : 0.667
('love', 'learning') : 0.333
('learning', 'python') : 1.0
('python', '.') : 1.0
('love', 'nlp') : 0.333
('nlp', '.') : 1.0

Laplace Smoothed Probabilities:
('i', 'love') : 0.333
('love', 'natural') : 0.167
('natural', 'language') : 0.2
('language', 'processing') : 0.2
('processing', '.') : 0.2
('.', 'i') : 0.25
('love', 'learning') : 0.167
('learning', 'python') : 0.2
('python', '.') : 0.2
('love', 'nlp') : 0.167
('nlp', '.') : 0.2
```

ANALYSIS AND INTERPRETATION

- Word frequency identifies the most commonly used words.
- Bigram frequency identifies frequently occurring word pairs.
- Unsmoothed probability becomes zero for unseen bigrams.
- Laplace smoothing assigns a small non-zero probability to unseen bigrams.
- Smoothing helps handle unknown word combinations.
- It improves the reliability of language models in text prediction and NLP applications.

RESULT

Thus, the program successfully calculates word and bigram frequencies and compares unsmoothed bigram probabilities with Laplace-smoothed probabilities.

QUESTION 8

AIM

To develop a Python program to calculate unigram and bigram probabilities and compute entropy using Shannon's entropy formula.

CONCEPT

Entropy is a measure of uncertainty or randomness in a text. In Natural Language Processing (NLP), it is used to measure how predictable words are in a language.

Shannon entropy is calculated using the formula:

$$H(X)=-\sum P(x)\log_2 P(x)$$

Unigram entropy measures uncertainty based on individual word probabilities, while bigram entropy considers probabilities of consecutive word pairs.

PROGRAM

```
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter
import nltk
import math
nltk.download('punkt_tab')
print("Enter a paragraph (press Enter twice to finish):")
lines = []
while True:
    line = input()
    if line == "":
        break
    lines.append(line)
text = " ".join(lines).lower()
words = word_tokenize(text)
unigram = Counter(words)
bigram = Counter(ngrams(words, 2))
total_words = sum(unigram.values())
total_bigrams = sum(bigram.values())
```

```
unigram_entropy = 0
bigram_entropy = 0
for count in unigram.values():
    p = count / total_words
    unigram_entropy -= p * math.log2(p)
for count in bigram.values():
    p = count / total_bigrams
    bigram_entropy -= p * math.log2(p)
print("\nUnigram Frequencies:", unigram)
print("Bigram Frequencies:", bigram)
print("\nUnigram Entropy:", round(unigram_entropy, 3))
print("Bigram Entropy:", round(bigram_entropy, 3))
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/
NLP AssignemntProg-8.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a paragraph (press Enter twice to finish):
I love natural language processing.
I love learning Python.
I love NLP.


Unigram Frequencies: Counter({'i': 3, 'love': 3, '.': 3, 'natural': 1,
'language': 1, 'processing': 1, 'learning': 1, 'python': 1, 'nlp': 1})
Bigram Frequencies: Counter({('i', 'love'): 3, ('.', 'i'): 2, ('love',
'natural'): 1, ('natural', 'language'): 1, ('language', 'processing'):
1, ('processing', '.'): 1, ('love', 'learning'): 1, ('learning', 'pytho
n'): 1, ('python', '.'): 1, ('love', 'nlp'): 1, ('nlp', '.'): 1})

Unigram Entropy: 2.956
Bigram Entropy: 3.325
```

ANALYSIS AND INTERPRETATION

- Unigram entropy measures the uncertainty of individual words.
- Bigram entropy considers the relationship between consecutive words.
- Higher entropy indicates greater uncertainty and less predictability.
- Lower entropy indicates more predictable word patterns.
- Bigram models provide contextual information that unigrams do not capture.
- Entropy is useful in language modelling, text prediction, and speech recognition.

RESULT

Thus, the program successfully calculates unigram and bigram entropy using Shannon's entropy formula and compares the uncertainty of both language models.

QUESTION 9

AIM

To develop a Python program to perform POS tagging on a given English paragraph, identify nouns, verbs, adjectives, adverbs and prepositions, and display their frequencies.

CONCEPT

Part-of-Speech (POS) tagging is the process of assigning grammatical categories to each word in a sentence based on its meaning and context.

Common POS categories include:

- Noun (NN): Names of people, places, or things.
- Verb (VB): Action or state words.
- Adjective (JJ): Describes a noun.
- Adverb (RB): Describes a verb or adjective.
- Preposition (IN): Shows relationships between words.

POS tagging helps computers understand sentence structure and grammatical patterns.

PROGRAM

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
text = input("Enter a paragraph: ")
words = word_tokenize(text)
tags = nltk.pos_tag(words)
print("\nPOS Tagging:")
for word, tag in tags:
    print(word, ":", tag)
categories = { "Nouns": ["NN", "NNS", "NNP", "NNPS"],"Verbs": ["VB", "VBD", "VBG", "VBN", "VBP", "VBZ"],"Adjectives": ["JJ", "JJR", "JJS"],"Adverbs": ["RB", "RBR", "RBS"],"Prepositions": ["IN"]}
print("\nPOS Category Frequencies:")
for category, tag_list in categories.items():
    count = sum(1 for word, tag in tags if tag in tag_list)
    print(category, ":", count)
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/NLP As
signemntProg-9.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\Hema Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter a paragraph: The intelligent student reads books carefully. She writes
Python programs and learns new technologies.

POS Tagging:
The : DT
intelligent : JJ
student : NN
reads : VBZ
books : NNS
carefully : RB
. : .
She : PRP
writes : VBZ
Python : NNP
programs : NNS
and : CC
learns : VBZ
new : JJ
technologies : NNS
. : .

POS Category Frequencies:
Nouns : 5
Verbs : 3
Adjectives : 2
Adverbs : 1
Prepositions : 0
```

ANALYSIS AND INTERPRETATION

- Each word is assigned an appropriate grammatical tag.
- Nouns identify people, places, and objects.
- Verbs represent actions or states.
- Adjectives describe nouns, while adverbs modify verbs or adjectives.
- Prepositions indicate relationships between words.
- POS tagging helps in syntactic analysis, information extraction, and machine translation.

RESULT

Thus, the program successfully performs POS tagging and displays the words with their corresponding tags and category frequencies.

QUESTION 10

AIM

To develop a Python program to implement and compare Rule-Based, Stochastic, and Transformation-Based POS tagging techniques using NLTK.

CONCEPT

POS tagging is the process of assigning grammatical labels to words in a sentence. There are three main approaches to POS tagging:

- Rule-Based Tagging: Uses predefined grammatical rules and dictionaries to assign tags.
- Stochastic Tagging: Uses statistical probabilities from training data to predict the most suitable tags.
- Transformation-Based Tagging: Starts with initial tags and improves them using learned correction rules.

These methods help identify grammatical structures and understand sentence patterns.

PROGRAM

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import RegexpTagger, UnigramTagger, BrillTaggerTrainer
from nltk.tag.brill import Template, Pos
nltk.download('punkt_tab')
nltk.download('brown')
text = input("Enter a sentence: ")
words = word_tokenize(text)
train_data = nltk.corpus.brown.tagged_sents(categories='news')[:2000]
rule_patterns = [
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD'),
    (r'.*ly$', 'RB'),
    (r'.*ous$', 'JJ'),
    (r'.*', 'NN')
]
```

```
rule_tagger = RegexpTagger(rule_patterns)
stochastic_tagger = UnigramTagger(train_data)
templates = [Template(Pos([-1])), Template(Pos([1])), Template(Pos([-1]), Pos([1]))]
trainer = BrillTaggerTrainer(stochastic_tagger, templates, trace=0)
brill_tagger = trainer.train(train_data, max_rules=10)
print("\nRule-Based Tagging:")
print(rule_tagger.tag(words))
print("\nStochastic Tagging:")
print(stochastic_tagger.tag(words))
print("\nTransformation-Based Tagging:")
print(brill_tagger.tag(words))
```

OUTPUT

```
= RESTART: C:/Users/Hema Neepa/AppData/Local/Programs/Python/Python313/N
LP AssignemntProg-10.py
[nltk_data] Downloading package punkt_tab to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package brown to C:\Users\Hema
[nltk_data]     Neepa\AppData\Roaming\nltk_data...
[nltk_data]   Package brown is already up-to-date!
Enter a sentence: The students are learning Python programming.

Rule-Based Tagging:
[('The', 'NN'), ('students', 'NN'), ('are', 'NN'), ('learning', 'VBG'),
('Python', 'NN'), ('programming', 'VBG'), ('.', 'NN')]

Stochastic Tagging:
[('The', 'AT'), ('students', 'NNS'), ('are', 'BER'), ('learning', 'VBG')
, ('Python', None), ('programming', None), ('.', '.')]

Transformation-Based Tagging:
[('The', 'AT'), ('students', 'NNS'), ('are', 'BER'), ('learning', 'VBG')
, ('Python', None), ('programming', None), ('.', '.')]
```

ANALYSIS AND INTERPRETATION

- Rule-based tagging depends on predefined linguistic rules.
- Stochastic tagging uses probability and training data to assign tags.
- Transformation-based tagging corrects initial tagging errors using learned rules.
- Rule-based methods are suitable for structured grammatical patterns.
- Stochastic methods handle language variations using statistical learning.
- Transformation-based methods combine initial tagging with rule-based corrections.
- Comparing these methods helps identify their effectiveness and accuracy in NLP applications.

RESULT

Thus, the program successfully implements and compares Rule-Based, Stochastic, and Transformation-Based POS tagging techniques using Python and NLTK.